

IoT & AI Based Real-Time Pothole Detection

Mini Project – IoT Case Study | CodeAlpha Internet of Things Internship

1. Introduction

Road-surface damage such as potholes is a practical transportation problem because road conditions can change over time and damaged sections may be difficult to identify through continuous manual inspection. Computer vision provides a way to observe road scenes using cameras, while Artificial Intelligence (AI) can be used to identify visual patterns associated with potholes. When this processing is performed on an embedded computer and connected to a communication system, it can form the basis of an IoT-enabled road monitoring solution.

This case study presents a student-developed prototype for real-time pothole detection using a camera, Raspberry Pi 4, Python, OpenCV, and a YOLO-based object-detection model. The main processing is performed locally on the Raspberry Pi, making the project an example of Edge AI. The detected information can also be used as an IoT event for logging, alert generation, and future connection to a remote monitoring platform.

2. Problem Statement

Traditional road inspection can require significant time and human effort. A vehicle-mounted camera can continuously capture road images while travelling, allowing software to examine successive frames. The engineering objective of this project is to develop a compact system that can detect potholes from live video, display the detected regions, and provide a foundation for connected road-condition monitoring.

3. Objectives

- Detect potholes in real time using computer vision and a YOLO-based model.
- Perform AI inference on a Raspberry Pi 4 edge-computing platform.
- Display detected potholes using bounding boxes and confidence values.
- Provide a basis for alerts, logging, location tagging, and future IoT connectivity.

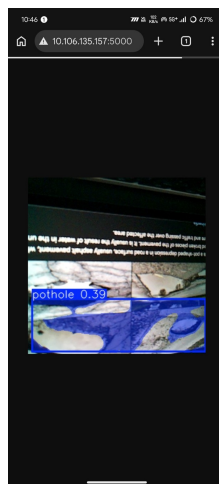


Figure 1. Project poster showing the proposed road-anomaly monitoring system and its implementation.

4. IoT and AI System Architecture

The proposed system combines sensing, edge computation, AI-based perception, and communication. A camera captures the road scene and supplies image frames to the Raspberry Pi 4. The Raspberry Pi performs the required image handling and passes the frame to the trained YOLO detector. When a pothole is identified, the model produces a detection containing a class label, confidence value, and bounding-box coordinates.

The AI component provides the visual recognition capability, while the IoT component provides a path for detection information to be stored, communicated, and used by another application. In a future deployment, a detection could be associated with a timestamp and GPS position and transmitted as a compact event to a remote database or dashboard.

5. Working Principle

1. The camera captures a live road image.
2. The Raspberry Pi receives and preprocesses the image frame.
3. The YOLO model performs object detection on the frame.
4. Detected potholes are highlighted with bounding boxes and confidence values.
5. Detection information can be displayed or recorded as a road-anomaly event.
6. A connected version can send selected events to a cloud/database or monitoring dashboard.

6. Edge AI Approach

YOLO is a real-time object-detection approach that predicts object locations and classes from an image [2]. It is suitable for this project because road monitoring requires repeated analysis of incoming frames. Running inference on the local Raspberry Pi reduces the need to continuously upload camera frames to a remote server for basic detection. Ultralytics also provides documentation for deploying YOLO models on Raspberry Pi and using optimized deployment formats [1].

The Raspberry Pi 4 provides a quad-core 64-bit processor together with USB, networking, wireless communication, GPIO, and camera interfaces [3]. These features make it suitable for an embedded prototype in which the camera, processing unit, and communication functions are placed close to the vehicle or monitoring device.

7. System Flow

Camera → Raspberry Pi 4 → Image Processing → YOLO Detection → Pothole Event → Display / Storage / Alert → IoT Dashboard

8. Hardware and Software Requirements

Hardware

- Raspberry Pi 4 – local processing and control platform.
- Camera module / USB camera – live road-image acquisition.
- Display and network connection – development, monitoring, and connectivity.

Software

- Python – application programming and inference control.
- OpenCV – image and video processing.
- YOLO-based model – pothole object detection.
- ONNX – model interchange/optimization option for edge deployment.
- Linux/Raspberry Pi software environment – execution platform.

9. Model Training and Detection

The project uses labelled pothole images to train a detector for the target class. During inference, each incoming frame is examined by the trained model. When the visual content matches the learned pothole class with sufficient confidence, the application displays a bounding box and confidence value. This makes it possible to see both the location of the detected region and the model's confidence for that particular detection.

The supplied development output shows inference running repeatedly on 320×320 input frames. The terminal output also shows changing detection counts between frames, demonstrating continuous inference during the prototype test.

10. Real-Time Prototype Results

The live test output supplied for this case study shows multiple potholes being detected simultaneously. Each detection is displayed with a rectangular bounding box and a confidence value. This demonstrates the intended real-time detection behaviour of the prototype.

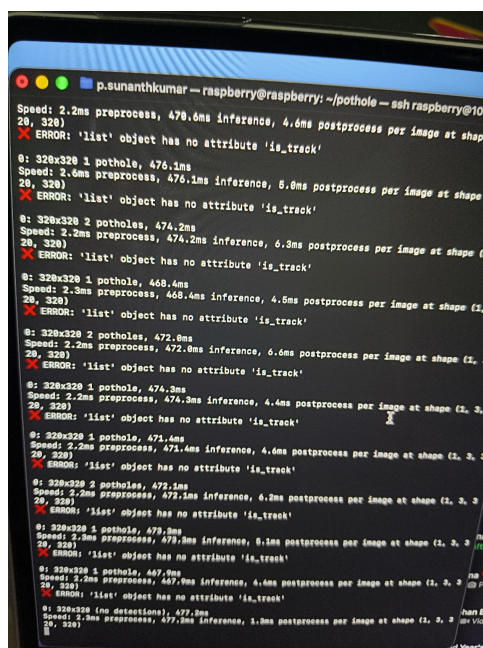


Figure 2. Live pothole-detection output from the project prototype.

11. Model Performance

The project materials include an evaluation graphic reporting Accuracy = 85, Precision = 82, and Recall = 80. These values are presented here as the results supplied with the project. They are not recalculated in this case study because the complete validation dataset and prediction records were not supplied.

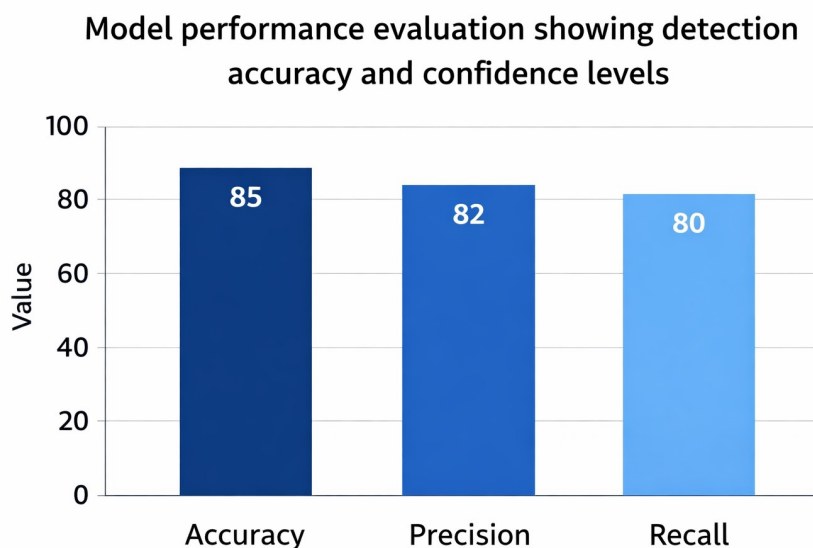


Figure 3. Project-provided model performance graphic showing the reported accuracy, precision, and recall values.

12. Discussion

The prototype demonstrates the complete path from image acquisition to AI-based detection. The Raspberry Pi performs the processing close to the camera, while the detection interface makes the result immediately visible. This edge-based approach is useful when a system needs to operate without depending on continuous cloud inference.

Detection performance can nevertheless vary with road appearance, lighting, camera position, motion blur, weather, and the difference between real-world potholes and the training examples. Model size and input resolution also influence the processing load on an embedded computer. Therefore, a production system would require testing across different roads, vehicles, lighting conditions, and weather conditions.

13. IoT Integration

The current prototype can be extended by treating every reliable pothole detection as a connected data event. A GPS module could provide location, while the Raspberry Pi could attach a timestamp and confidence value. The system could then send a small record to a server instead of continuously transmitting the complete video stream.

A cloud or local server could store these records and display them on a dashboard. Repeated detections from different journeys could be grouped by road segment, helping create a digital map of observed road anomalies. This extension would connect the AI perception layer to the monitoring and data-management functions of an IoT system.

14. Applications

- Road-condition surveys using municipal or maintenance vehicles.
- Driver-assistance and connected-vehicle research.
- Smart-city infrastructure monitoring.
- Mapping frequently detected pothole locations.
- Supporting road-maintenance inspection and planning.

15. Advantages

The system can provide continuous visual monitoring, reduce dependence on manual observation, and process detections locally. The use of an embedded computer makes the prototype portable and suitable for vehicle-based testing. The addition of wireless communication can further turn individual detections into a connected road-monitoring dataset.

16. Limitations

The system's performance depends on the quality and variety of its training data. Difficult lighting, rain, shadows, damaged road surfaces, camera vibration, and unusual pothole shapes may affect detection. The Raspberry Pi also has less computational capacity than a desktop GPU, so model size and image resolution must be selected carefully. These factors should be considered before moving from a prototype to large-scale deployment.

17. Future Scope

Future work can include GPS-based location tagging, a web dashboard, cloud or local database storage, better filtering of low-confidence detections, and tracking the same pothole across consecutive video frames. The detector can also be expanded to additional road hazards such as debris, animals, or stalled vehicles if appropriate labelled datasets are available.

Model optimization can be explored to improve inference speed on Raspberry Pi hardware. A connected fleet of vehicles could also contribute observations to a shared road-condition database, allowing repeated detections to be compared over time.

18. Conclusion

This project demonstrates an IoT-oriented Edge AI approach to real-time pothole detection. A camera supplies road images, the Raspberry Pi 4 performs local processing, and a YOLO-based model identifies potholes and displays them with bounding boxes and confidence values. The supplied prototype results demonstrate the intended live-detection operation.

With GPS, networking, data storage, and a monitoring dashboard, the system can be developed into a connected road-condition monitoring platform. The project therefore illustrates how AI-based computer vision and IoT technologies can be combined in an embedded system for smart transportation applications.

References

- [1] Ultralytics. *YOLO on Raspberry Pi – Edge Deployment Guide*. Official documentation. <https://docs.ultralytics.com/guides/raspberry-pi>
- [2] Redmon, J., Divvala, S., Girshick, R., & Farhadi, A. (2016). "You Only Look Once: Unified, Real-Time Object Detection." *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 779–788. <https://doi.org/10.1109/CVPR.2016.91>
- [3] Raspberry Pi. *Raspberry Pi 4 Model B – Technical Specifications*. Official documentation. <https://www.raspberrypi.com/products/raspberry-pi-4-model-b/specifications/>
- [4] Ultralytics. *Model Deployment Options*. Official documentation. <https://docs.ultralytics.com/platform/deploy>
- [5] Redmon, J., & Farhadi, A. (2016). "YOLO9000: Better, Faster, Stronger." <https://arxiv.org/abs/1612.08242>